

Chapitre 2: Les algorithmes de détection de communautés dans les réseaux sociaux

Master: Sciences de données pour une industrie intelligente

Pr. Elyazid AKACHAR

Email: elyazid.akachar@gmail.com

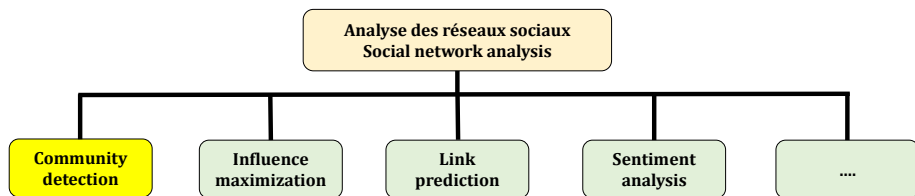
- ➊ Introduction
- ➋ Détection de communautés
- ➌ Qu'est-ce qu'une communauté ?
- ➍ Pourquoi détecter des communautés ?
- ➎ Types de communautés dans les réseaux sociaux
- ➏ Approches pour la détection de communautés
- ➐ Modularité
- ➑ Algorithme de Girvan–Newman (GN)
- ➒ Algorithme de Louvain
- ➓ Méthode de propagation d'étiquettes : Label propagation
- ➔ Mesures de qualité pour la détection de communautés
- ➕ Détection de communautés dans les réseaux sociaux dynamiques

Introduction

✍ La Social Network Analysis (SNA) est un domaine de l'informatique qui vise à extraire des informations à partir des réseaux sociaux, qu'ils soient en ligne ou physiques, en étudiant les relations et les interactions entre les individus, groupes ou organisations au sein d'un réseau. Plutôt que de se concentrer uniquement sur les caractéristiques des individus, elle s'intéresse à la structure des liens qui les relient.

✍ En d'autres termes, la SNA examine qui est connecté à qui, comment l'information circule, et comment les structures relationnelles influencent les comportements, les opinions ou la diffusion d'informations.

✍ La Social Network Analysis (SNA) est constituée de plusieurs sous-domaines, chacun se concentrant sur un aspect spécifique de l'étude des réseaux.



Détection de communautés

Les plateformes de réseaux sociaux favorisent l'émergence de groupes structurés autour d'intérêts communs – musique, sport, cinéma, politique, etc. Ces groupes peuvent exister naturellement, en se formant à partir des affinités et des interactions entre utilisateurs, ou être créés directement par les utilisateurs.

La détection de communautés est le domaine qui permet d'identifier automatiquement ces groupes d'utilisateurs partageant des intérêts similaires au sein d'un réseau social, en se basant sur leurs interactions et leurs affinités.

La détection de communautés est le processus qui consiste à identifier, pour chaque utilisateur d'un réseau social, le groupe avec lequel il interagit le plus.

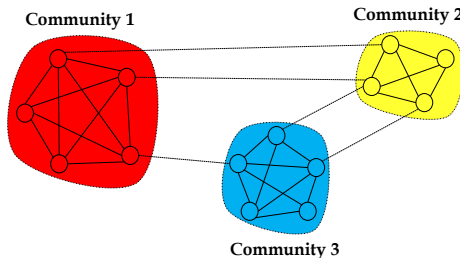


Qu'est-ce qu'une communauté ?

Dans le contexte des réseaux sociaux, une **communauté** désigne un ensemble d'utilisateurs fortement connectés entre eux, et faiblement connectés au reste du réseau. Les membres d'une même communauté interagissent fréquemment et partagent souvent des centres d'intérêt communs, ce qui renforce leur cohésion interne.

📌 Exemples :

- un groupe d'amis proches sur Facebook
- un cercle familial sur WhatsApp
- un groupe de fans autour d'un même artiste sur Instagram
- des chercheurs d'un même domaine qui se citent entre eux



Pourquoi détecter des communautés ?

✍ La détection de communautés permet de mieux comprendre la structure d'un réseau et d'extraire une information utile.

✍ la détection de communautés permet de révéler la logique cachée derrière les échanges dans un réseau.

Principaux objectifs :

- Comprendre l'organisation d'un réseau social (qui interagit avec qui).
- Identifier les groupes d'intérêt ou de discussions (utile en marketing ou en communication).
- Analyser la diffusion de l'information : actualités, rumeurs, fake news...
- Recommander des contenus personnalisés (amis, annonces, vidéos, articles).
- Repérer des comportements anormaux (bots, fraude, spam).

Pourquoi détecter des communautés ?

🔗 Dans tous les cas, les plateformes utilisent des algorithmes de détection de communautés pour structurer, analyser et recommander du contenu.

❶ **Facebook / Instagram** : recommander des contenus, pages ou groupes adaptés aux centres d'intérêt des utilisateurs (sport, cuisine, voyage...).

➡ **Par exemple** : proposer à un utilisateur passionné de cuisine de rejoindre un groupe de recettes.

❷ **Twitter / X** : analyser la diffusion des opinions et des tendances dans des communautés thématiques (politique, foot, crypto...).

➡ **Par exemple** : comprendre comment une idée politique se propage d'une communauté à une autre.

❸ **LinkedIn** : identifier les communautés professionnelles pour améliorer les recommandations d'emploi ou de réseau.

➡ **Par exemple** : suggérer des offres d'emploi à un ingénieur ou des connexions pertinentes à un enseignant-chercheur.

❹ **YouTube** : recommander automatiquement des vidéos basées sur les habitudes de visualisation des utilisateurs.

➡ **Par exemple** : proposer plus de tutoriels de bricolage à quelqu'un qui en regarde régulièrement.

Types de communautés dans les réseaux sociaux

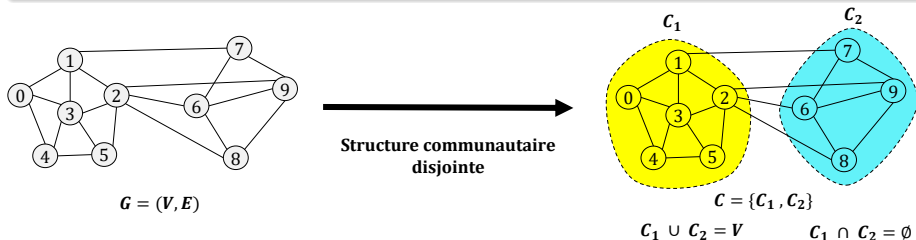
✎ Dans les réseaux sociaux, les communautés peuvent être classées en deux types principaux: les **communautés disjointes** et les **communautés chevauchantes**, selon la manière dont les utilisateurs sont répartis entre les groupes.

✎ Une **structure communautaire disjointe** est une répartition d'un réseau social dans laquelle chaque utilisateur appartient à une seule communauté.

✎ Soit $G = (V, E)$ un graphe représentant un réseau social, où V est l'ensemble des nœuds (utilisateurs) et E l'ensemble des arêtes (relations). Une **structure communautaire disjointe** du graphe G est un ensemble $C = \{C_1, C_2, \dots, C_k\}$ de sous-ensembles de V tel que :

- ❶ $\bigcup_{i=1}^k C_i = V$ (tous les nœuds appartiennent à une communauté)
- ❷ $C_i \cap C_j = \emptyset$, pour tout $i \neq j$ (les communautés sont mutuellement exclusives)

Chaque C_i correspond à une **communauté**, et chaque nœud $v \in V$ appartient à **exactement une communauté**.



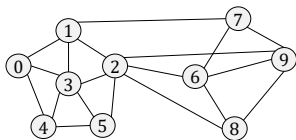
Types de communautés dans les réseaux sociaux

✎ Une **structure communautaire chevauchante** est une répartition d'un réseau social dans laquelle un utilisateur peut appartenir à plusieurs communautés.

Soit $G = (V, E)$ un graphe représentant un réseau social, où V est l'ensemble des nœuds (utilisateurs) et E l'ensemble des arêtes (relations). Une **structure communautaire chevauchante** du graphe G est un ensemble $C = \{C_1, C_2, \dots, C_k\}$ de sous-ensembles de V tel que :

- 1 $\bigcup_{i=1}^k C_i = V$ (tous les nœuds appartiennent à au moins une communauté)
- 2 $C_i \cap C_j \neq \emptyset$ est possible pour certains $i \neq j$ (les communautés peuvent se chevaucher)

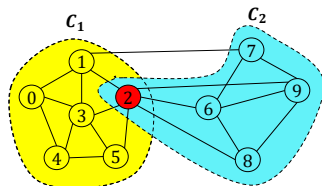
Chaque C_i correspond à une **communauté**, et un nœud $v \in V$ peut appartenir à **plusieurs communautés**.



$G = (V, E)$



Structure communautaire
chevauchante



$C = \{C_1, C_2\}$

$$C_1 \cup C_2 = V$$

$$C_1 \cap C_2 = \{2\}$$

✍ Le problème de la détection de communautés a été largement étudié, et de nombreux algorithmes ont été développés. Chaque solution repose sur un mécanisme ou une approche spécifique. Parmi les méthodes les plus connues dans la littérature, on distingue notamment :

- Les algorithmes basés sur la modularité
- Les algorithmes basés sur la coupure du graphe (Graph Partitioning)
- Les algorithmes hiérarchiques (agglomérative (bottom-up) , divisive (top-down))
- Les algorithmes basés sur la propagation d'étiquettes
- Les algorithmes probabilistes et modèles statistiques
- Les algorithmes basés sur des seeds (graine)

- ✍ La modularité est une mesure introduite et popularisée par Mark Newman en 2002 pour l'analyse de la structure communautaire dans les réseaux.
- ✍ Elle permet d'évaluer la qualité d'une partition du réseau en mesurant si les nœuds d'une même communauté sont plus connectés entre eux que ce que l'on attendrait au hasard.
- ✍ Ce score reflète le niveau de densité interne d'une communauté par rapport à une configuration aléatoire.
- ✍ **Principe de la modularité :**

Le principe de la modularité repose sur une idée fondamentale : **Une bonne communauté est un groupe où les liens internes sont plus nombreux que ce que l'on obtiendrait par hasard.**

Autrement dit, la modularité compare deux quantités :

- **La réalité :**

➡ le nombre de liens réellement présents à l'intérieur d'une communauté C_m . Ce terme est calculé par :

$$\sum_{i,j \in C_m} A_{ij}$$

où A_{ij} est un élément de la matrice d'adjacence : $A_{ij} = 1$ s'il y a un lien entre les nœuds i et j , et 0 sinon.

- **L'attendu par hasard :**

➡ Le nombre de liens que l'on s'attendrait à trouver à l'intérieur d'une communauté si les connexions du réseau étaient distribuées de manière aléatoire, tout en conservant les degrés des nœuds. Ce terme correspond à la probabilité qu'un lien existe entre chaque paire de nœuds (i, j) dans ce modèle aléatoire. Cette probabilité s'obtient par la formule:

$$\sum_{i,j \in C_m} \frac{d_i \cdot d_j}{2m}$$

où d_i est le degré du nœud i , et m représente le nombre total de liens dans le graphe.

$$\text{Modularité (Q)} = (\text{Liens réels à l'intérieur des communautés}) - (\text{Liens attendus au hasard})$$

✎ Puisque la modularité compare les liens observés aux liens attendus dans un modèle aléatoire, la modularité d'une communauté C_k s'exprime par :

$$Q_{C_k} = \frac{1}{2m} \sum_{i,j \in C_k} A_{ij} - \frac{1}{2m} \sum_{i,j \in C_k} \frac{d_i \cdot d_j}{2m}$$

$$Q_{C_k} = \frac{1}{2m} \sum_{i,j \in C_k} \left(A_{ij} - \frac{d_i \cdot d_j}{2m} \right)$$

✎ La somme des liens réels et la somme des liens attendus peuvent atteindre des valeurs très élevées, surtout dans les grands réseaux. Pour éviter que la modularité ne devienne trop grande et pour assurer une valeur normalisée comprise entre -1 et 1 , il est nécessaire de diviser l'ensemble par $2m$.

✎ La modularité totale d'une partition $C = \{C_1, C_2, \dots, C_p\}$ du graphe est donnée par la somme des modularités individuelles de chaque communauté :

$$Q(C) = \sum_{k=1}^p Q_{C_k} = \frac{1}{2m} \sum_{k=1}^p \sum_{i,j \in C_k} \left(A_{ij} - \frac{d_i \cdot d_j}{2m} \right)$$

✎ On peut remplacer la double somme sur les communautés par une seule somme sur toutes les paires de nœuds, en ajoutant une fonction indicatrice $\delta(c_i, c_j)$ qui vaut :

$$\delta(c_i, c_j) = \begin{cases} 1, & \text{si } c_i = c_j \text{ (i et j appartiennent à la même communauté)} \\ 0, & \text{sinon} \end{cases}$$

✎ La modularité peut alors s'écrire sous la forme :

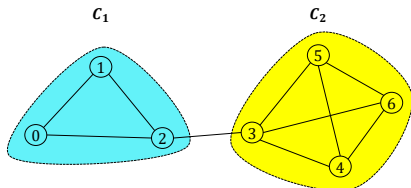
$$Q = \frac{1}{2m} \sum_{i,j \in V} \left(A_{ij} - \frac{d_i d_j}{2m} \right) \delta(c_i, c_j)$$

✎ La modularité mesure la “force” des communautés dans le réseau:

- Modularité Q proche de 1 $\rightarrow$ très bonne partition
- Modularité Q proche de 0 $\rightarrow$ partition faible
- Modularité Q négative $\rightarrow$ mauvaise partition

✎ **Exemple :** On considère le graphe $G : G = [(0, 1), (0, 2), (1, 2), (2, 3), (3, 4), (3, 5), (3, 6), (4, 5), (4, 6), (5, 6)]$, et la partition suivante : $S = [\{0, 1, 2\}, \{3, 4, 5, 6\}]$.

Calculer la modularité de C_1 et C_2 puis déduire la modularité de la partition S



- Communauté : $C_1 = \{0, 1, 2\}$
- Nombre total de liens : $m = 10$
- Degrés : $d_0 = 2, d_1 = 2, d_2 = 3$

Formule pour une communauté :

$$Q_{C_1} = \frac{1}{2m} \sum_{i,j \in C_1} \left(A_{ij} - \frac{d_i d_j}{2m} \right)$$

Différence entre liens réels et liens attendus aléatoirement :

(i, j)	A_{ij}	$d_i d_j / 2m$	$A_{ij} - d_i d_j / 2m$
(0, 0)	0	0.20	-0.20
(0, 1)	1	0.20	0.80
(0, 2)	1	0.30	0.70
(1, 0)	1	0.20	0.80
(1, 1)	0	0.20	-0.20
(1, 2)	1	0.30	0.70
(2, 0)	1	0.30	0.70
(2, 1)	1	0.30	0.70
(2, 2)	0	0.45	-0.45

Somme interne :

$$-0.2 + 0.8 + 0.7 + 0.8 - 0.2 + 0.7 + 0.7 + 0.7 - 0.45 = 3.55$$

Normalisation :

$$Q_{C_1} = \frac{3.55}{2m} = \frac{3.55}{20} = 0.1775$$

- Communauté : $C_2 = \{3, 4, 5, 6\}$
- Nombre total de liens : $m = 10$
- Degrés : $d_3 = 4, d_4 = 3, d_5 = 3, d_6 = 3$

Formule :

$$Q_{C_2} = \frac{1}{2m} \sum_{i,j \in C_2} \left(A_{ij} - \frac{d_i d_j}{2m} \right)$$

Différence entre liens réels et liens attendus aléatoirement :

(i, j)	A_{ij}	$d_i d_j / 2m$	$A_{ij} - d_i d_j / 2m$
(3, 3)	0	0.80	-0.80
(3, 4)	1	0.60	0.40
(3, 5)	1	0.60	0.40
(3, 6)	1	0.60	0.40
(4, 3)	1	0.60	0.40
(4, 4)	0	0.45	-0.45
(4, 5)	1	0.45	0.55
(4, 6)	1	0.45	0.55
(5, 3)	1	0.60	0.40
(5, 4)	1	0.45	0.55
(5, 5)	0	0.45	-0.45
(5, 6)	1	0.45	0.55
(6, 3)	1	0.60	0.40
(6, 4)	1	0.45	0.55
(6, 5)	1	0.45	0.55
(6, 6)	0	0.45	-0.45

Somme interne :

$$-0.80 + 0.40 + 0.40 + 0.40 + 0.40 - 0.45 + 0.55 + 0.55 + 0.40 + 0.55 - 0.45 + 0.55 + 0.40 + 0.55 + 0.55 - 0.45 = 3.55$$

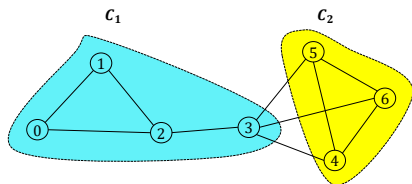
Normalisation :

$$Q_{C_2} = \frac{3.55}{2m} = \frac{3.55}{20} = 0.1775$$

Modularité totale de la partition :

$$Q = Q_{C_1} + Q_{C_2} = 0.1775 + 0.1775 = 0.355$$

✎ **Exemple :** On considère le même graphe de l'exemple précédent mais maintenant avec une partition $S_1 = [\{0, 1, 2, 3\}, \{4, 5, 6\}]$ différente de S . L'objectif est de calculer la modularité de S_1 et de la comparer avec celle de S .



- $m = 10$ liens
- Degrés: $d_0 = 2, d_1 = 2, d_2 = 3, d_3 = 4, d_4 = 3, d_5 = 3, d_6 = 3$
- Partition: $S_1 = [\{0, 1, 2, 3\}, \{4, 5, 6\}]$

✎ Calcul pour la communauté $C_2 = \{4, 5, 6\}$

(i, j)	A_{ij}	$d_i d_j / 2m$	$A_{ij} - d_i d_j / 2m$
(4, 4)	0	0.45	-0.45
(4, 5)	1	0.45	0.55
(4, 6)	1	0.45	0.55
(5, 4)	1	0.45	0.55
(5, 5)	0	0.45	-0.45
(5, 6)	1	0.45	0.55
(6, 4)	1	0.45	0.55
(6, 5)	1	0.45	0.55
(6, 6)	0	0.45	-0.45

✎ **Somme interne :**

$$\text{Somme}(C_2) = -0.45 + 0.55 + 0.55 + 0.55 - 0.45 + 0.55 + 0.55 + 0.55 - 0.45 = 1.95$$

✎ Calcul pour la communauté $C_1 = \{0, 1, 2, 3\}$

(i, j)	A_{ij}	$d_i d_j / 2m$	$A_{ij} - d_i d_j / 2m$
(0, 0)	0	0.20	-0.20
(0, 1)	1	0.20	0.80
(0, 2)	1	0.30	0.70
(0, 3)	0	0.40	-0.40
(1, 0)	1	0.20	0.80
(1, 1)	0	0.20	-0.20
(1, 2)	1	0.30	0.70
(1, 3)	0	0.40	-0.40
(2, 0)	1	0.30	0.70
(2, 1)	1	0.30	0.70
(2, 2)	0	0.45	-0.45
(2, 3)	1	0.60	0.40
(3, 0)	0	0.40	-0.40
(3, 1)	0	0.40	-0.40
(3, 2)	1	0.60	0.40
(3, 3)	0	0.80	-0.80

✎ Somme interne :

$$Somme(C_1) = (0.80 + 0.70 + 0.80 + 0.70 + 0.70 + 0.70 + 0.40 + 0.40) - (0.20 + 0.40 + 0.20 + 0.40 + 0.45 + 0.40 + 0.40 + 0.80) = 1.95$$

✎ Modularité totale

$$Q(S_1) = \frac{1}{2 \times 10} \times (1.95 + 1.95) = \frac{3.9}{20} = 0.195$$

✎ Comparaison avec la partition S

$$Q(S) = 0.355 > Q(S_1) = 0.195$$

La partition S a une modularité plus élevée, confirmant qu'elle est meilleure que S_1 .

✎ La modularité mentionnée précédemment peut s'exprimer sous une forme compacte, plus adaptée au calcul numérique.

$$Q = \frac{1}{2m} \sum_{i,j \in V} \left(A_{ij} - \frac{d_i d_j}{2m} \right) \delta(c_i, c_j) \quad \Leftrightarrow \quad Q = \sum_{c=1}^p \left[\frac{l_c}{m} - \left(\frac{D_c}{2m} \right)^2 \right]$$

Où :

- p = nombre de communautés
- l_c = nombre de liens **internes** à la communauté c
- D_c = somme des degrés dans la communauté c
- m = nombre total de liens

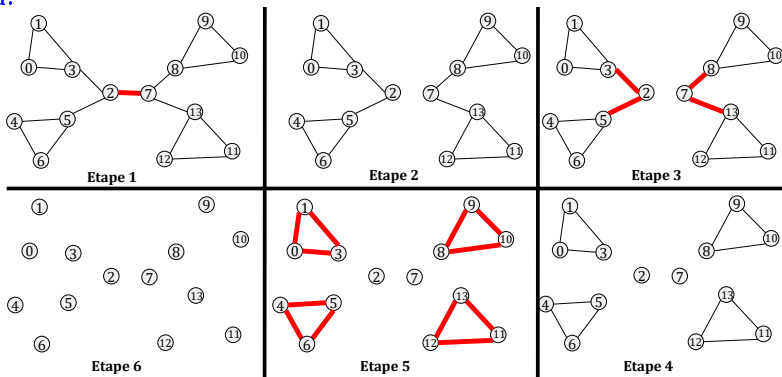
$$\begin{aligned} Q(S) &= \left[\frac{3}{10} - \left(\frac{7}{20} \right)^2 \right] + \left[\frac{6}{10} - \left(\frac{13}{20} \right)^2 \right] & Q(S_1) &= \left[\frac{4}{10} - \left(\frac{11}{20} \right)^2 \right] + \left[\frac{3}{10} - \left(\frac{9}{20} \right)^2 \right] \\ &= (0.3 - 0.1225) + (0.6 - 0.4225) & &= (0.4 - 0.3025) + (0.3 - 0.2025) \\ &= 0.1775 + 0.1775 & &= 0.0975 + 0.0975 \\ \Rightarrow \boxed{Q(S) = 0.355.} & & \Rightarrow \boxed{Q(S_1) = 0.195.} \end{aligned}$$

Algorithme de Girvan–Newman (GN)

✎ L'algorithme Girvan–Newman (GN) est une méthode hiérarchique divisive très connue pour la détection de communautés dans les graphes sociaux, proposée par Girvan et Newman en 2002.

✎ Il commence par considérer le graphe entier comme une seule communauté, puis il procède à des divisions successives pour séparer progressivement le réseau en plusieurs groupes, jusqu'à obtenir des communautés composées d'un seul nœud.

✎ **L'objectif de l'algorithme Girvan–Newman est de détecter les communautés en supprimant progressivement les liens ponts qui relient les différents groupes du réseau.**



Algorithme de Girvan–Newman (GN) : Étapes

🔗 Les étapes de GN algorithm :

- ➊ Calculer la betweenness de toutes les arêtes.
- ➋ Supprimer l'arête la plus centrale.
- ➌ Observer la nouvelle partition et calculer la modularité.
- ➍ Répéter le processus jusqu'à ce que toutes les arêtes soient supprimées, puis sélectionner la partition présentant la modularité la plus élevée.

➡ Étape ➊ : Calculer la betweenness des arêtes.

Pour chaque arête e du graphe, on souhaite connaître sa **edge betweenness** $B(e)$: le nombre (ou la somme pondérée) de plus courts chemins entre toutes les paires de nœuds (s, t) qui passent par e . Une arête ayant une valeur $B(e)$ élevée est généralement un **pont** entre communautés.

Formellement, la centralité d'intermédierité d'une arête e est définie par :

$$B(e) = \sum_{s \neq t} \frac{\sigma_{st}(e)}{\sigma_{st}},$$

où σ_{st} est le nombre de plus courts chemins entre s et t , et $\sigma_{st}(e)$ est le nombre de ces chemins qui traversent l'arête e .

Algorithme de Girvan–Newman (GN) : Étapes



Les arêtes du graphe sont : $(0, 1)$, $(1, 2)$ et $(2, 3)$.

🔗 **Paires de sommets** : Nous considérons toutes les paires de sommets distincts :

$(0, 1)$, $(0, 2)$, $(0, 3)$, $(1, 2)$, $(1, 3)$, $(2, 3)$.

🔗 **Plus courts chemins et contributions**

Pour chaque paire (s, t) , nous utilisons son plus court chemin et ajoutons une contribution 1 aux arêtes appartenant à ce chemin.

- $(0, 1)$: chemin $0 - 1 \Rightarrow (0, 1)+ = 1$
- $(0, 2)$: chemin $0 - 1 - 2 \Rightarrow (0, 1)+ = 1, (1, 2)+ = 1$
- $(0, 3)$: chemin $0 - 1 - 2 - 3 \Rightarrow (0, 1)+ = 1, (1, 2)+ = 1, (2, 3)+ = 1$
- $(1, 2)$: chemin $1 - 2 \Rightarrow (1, 2)+ = 1$
- $(1, 3)$: chemin $1 - 2 - 3 \Rightarrow (1, 2)+ = 1, (2, 3)+ = 1$
- $(2, 3)$: chemin $2 - 3 \Rightarrow (2, 3)+ = 1$

🔗 **Somme des contributions** :

$$\text{betw}(0, 1) = 1 + 1 + 1 = 3,$$

$$\text{betw}(1, 2) = 1 + 1 + 1 + 1 = 4,$$

$$\text{betw}(2, 3) = 1 + 1 + 1 = 3.$$

🔗 **Résultat final** :

Arête	Edge betweenness
$(0, 1)$	3
$(1, 2)$	4
$(2, 3)$	3

On observe que l'arête centrale $(1, 2)$ possède la valeur la plus élevée.

Algorithme de Girvan–Newman (GN) : Étapes

➡ **Étape ②** : Supprimer l'arête la plus centrale.

🔗 Après avoir calculé la centralité d'intermédiarité des arêtes (edge betweenness), l'algorithme passe à la deuxième étape, qui correspond à l'identification puis à la suppression de l'arête la plus centrale.

🔗 L'arête la plus centrale est celle qui :

- apparaît dans le plus grand nombre de plus courts chemins entre les sommets du graphe;
- possède la valeur de betweenness la plus élevée.

🔗 Cette arête est généralement un pont entre deux groupes du réseau. C'est pourquoi, dans la méthode de Girvan–Newman, on commence par supprimer cette arête.

🔗 Dans l'exemple précédent, on supprime l'arête (1,2) car c'est celle qui possède la plus grande valeur de betweenness, et donc elle joue le rôle de lien principal entre les deux parties du graphe.



🔗 Si plusieurs arêtes ont la même betweenness maximale, on peut les supprimer toutes d'un coup. Mais dans la version standard de l'algorithme Girvan–Newman, on retire seulement une arête à la fois, puis on recalcule les betweenness.

Algorithme de Girvan–Newman (GN) : Étapes

➡ **Étape ③** : Observer la nouvelle partition et calculer la modularité.

✎ Après la suppression d'une arête, on observe comment le graphe s'est découpé. Cela peut donner, par exemple :

- **1 communauté** : si le graphe reste connecté ;
- **2 communautés** : si une arête-pont a été retirée ;
- **plusieurs communautés** : si plusieurs zones se séparent.

✎ Après la suppression d'une arête, on calcule la modularité de la nouvelle division du graphe, puis on conserve la division qui possède la valeur de modularité la plus élevée.

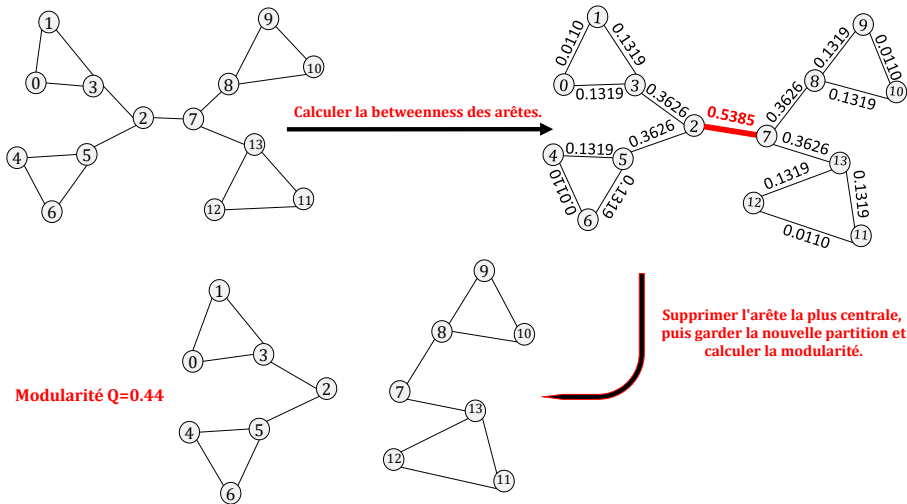
✎ Certaines implémentations enregistrent seulement toutes les partitions obtenues, puis choisissent à la fin celle qui a la plus grande modularité.

Étape ④ : Répéter le processus.

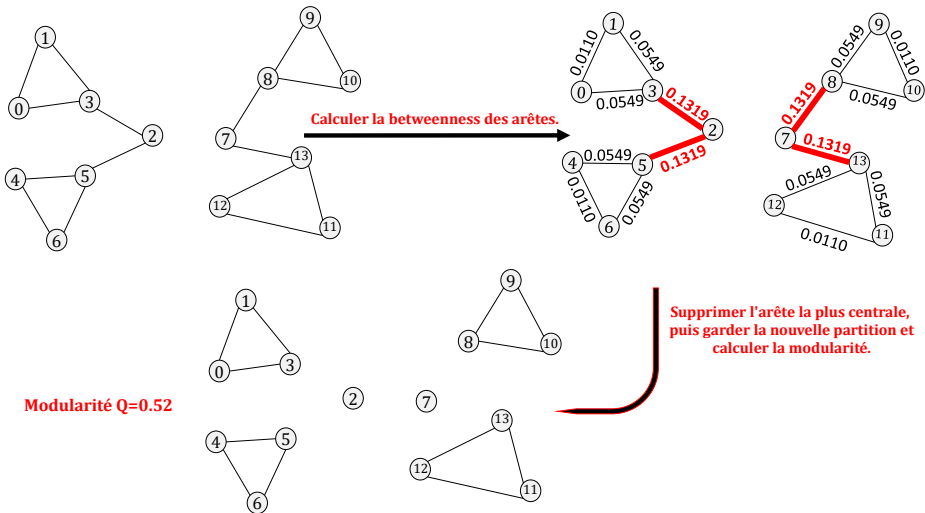
✎ On répète les étapes précédentes : recalculer la betweenness, supprimer l'arête la plus centrale, observer la nouvelle partition et évaluer sa modularité. La division qui maximise la modularité est ensuite retournée comme la structure communautaire détectée par l'algorithme de Girvan–Newman.

Algorithme de Girvan–Newman (GN) : Exemple

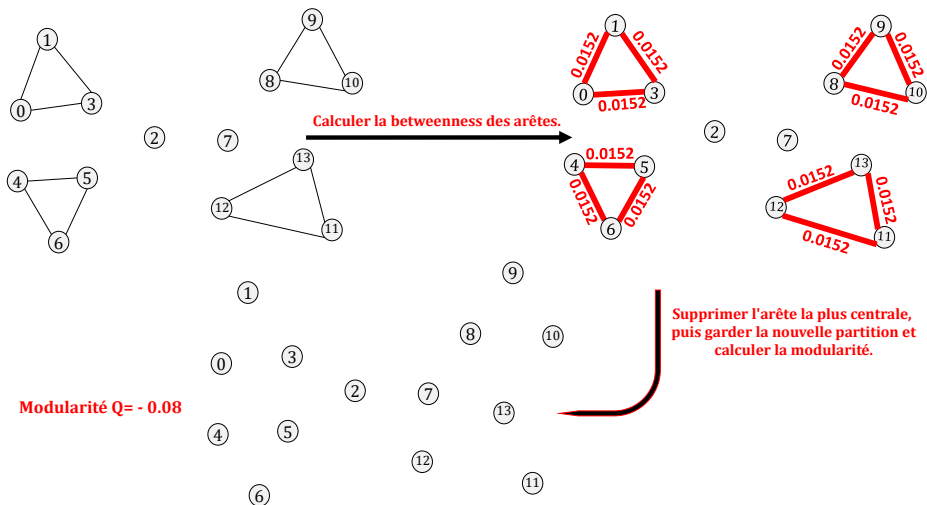
🔧 Dans cet exemple, nous avons utilisé l'implémentation de GN fournie par le package NetworkX. Cette implémentation supporte la suppression des arêtes (avec la même valeur de centralité) dans la même itération.



Algorithme de Girvan–Newman (GN) : Exemple



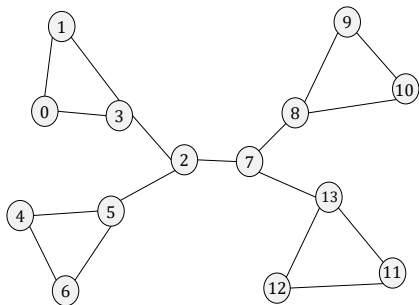
Algorithme de Girvan–Newman (GN) : Exemple



✎ Aucune arête ne reste dans le graphe, donc GN se termine.

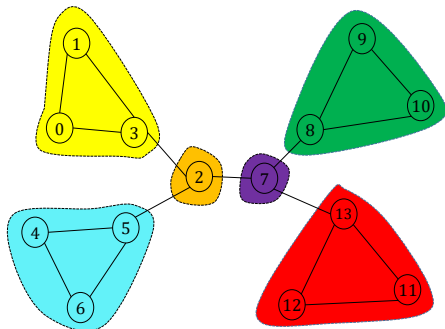
✎ La partition retenue par l'algorithme GN est celle qui maximise la modularité.

Algorithme de Girvan–Newman (GN) : Exemple



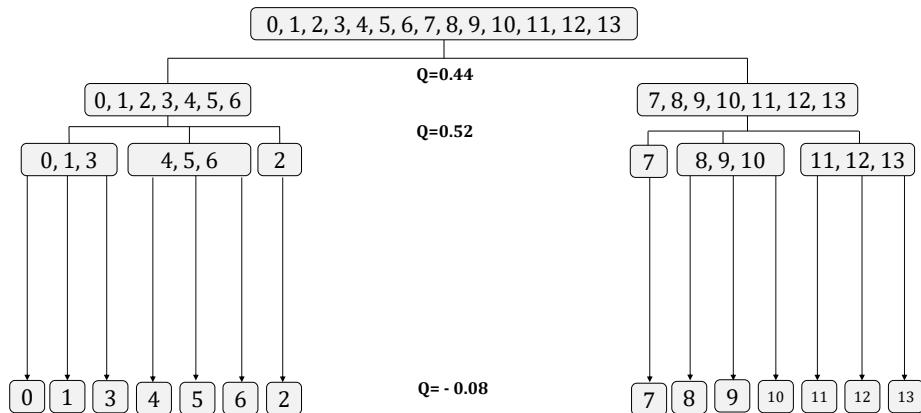
Structure communautaire détectée par l'algorithme de GN

$Q=0.52$



Algorithme de Girvan–Newman (GN) : dendrogramme

✍️ Puisque l'algorithme Girvan–Newman est un algorithme hiérarchique, on peut représenter ses différentes étapes de division sous la forme d'un arbre. En clustering de manière générale, ce type d'arbre est appelé un **dendrogramme**.



Algorithme de Girvan–Newman (GN) : dendrogramme



🔖 Avantages de l'algorithme Girvan–Newman (GN)

- Facile à comprendre
- Produit une hiérarchie de communautés
- Pas besoin de fixer le nombre de communautés
- Résultats fiables pour les petits graphes

🔖 Inconvénients de l'algorithme Girvan–Newman (GN)

- Très lent pour les grands graphes (Pas adapté aux réseaux réels de grande taille) $O(n.m^2)$
- Sensible aux arêtes ayant la même betweenness
- Peut produire des communautés déséquilibrées (communautés invalide)

✍ L'algorithme de Louvain, proposé par Vincent D. Blondel et ses collègues en 2008, porte ce nom en référence à l'Université catholique de Louvain (UCLouvain), où les auteurs étaient affiliés lors du développement de cette méthode.

✍ Louvain est un algorithme hiérarchique agglomératif (bottom-up) basé sur l'optimisation de la modularité.

✍ Au départ, chaque nœud est considéré comme une communauté, puis les communautés sont fusionnées uniquement si cette fusion optimise la modularité.

✍ Louvain fonctionne en deux grandes étapes, répétées de manière itérative :

① **Étape 1 : Optimisation locale de la modularité**

② **Étape 2 : Agrégation (compression du graphe)**

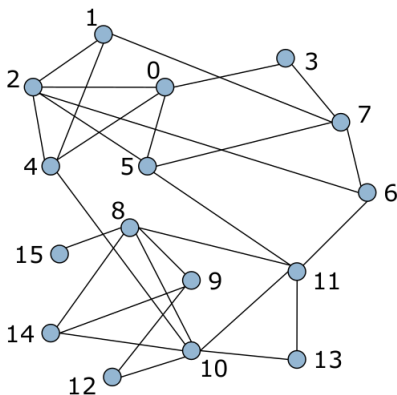
✍ Les deux phases sont répétées jusqu'à ce que la modularité n'augmente plus.

Algorithme de Louvain : Étapes

Phase ① : Optimisation locale de la modularité

✎ La première phase de l'algorithme de Louvain est constituée de plusieurs étapes.

① Au départ, chaque nœud est une communauté. Il y a autant de communautés que de nœuds dans le graphe.



➊ **Fusion des communautés** : pour chaque nœud i du graphe on considère chacun de ses voisins j et l'on évalue le gain de modularité que l'on obtiendrait en retirant i de sa communauté pour le mettre dans celle de j . Le nœud i est ensuite placé dans la communauté pour laquelle le gain de modularité est maximal, à condition que ce gain soit positif (sinon, i reste dans sa communauté).

➋ Pour un nœud i et une communauté C , le gain de modularité si i est déplacé dans C est :

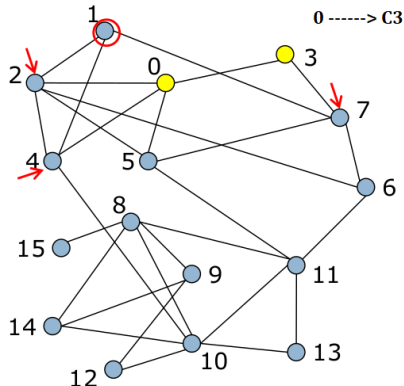
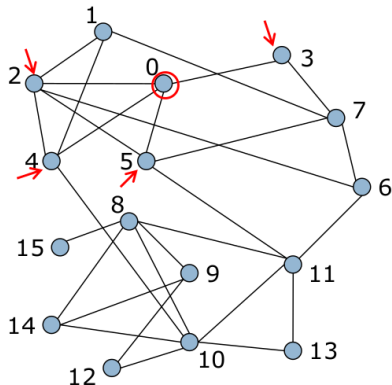
$$\Delta Q = \left[\frac{\Sigma_{\text{in}} + k_{i,\text{in}}}{2m} - \left(\frac{\Sigma_{\text{tot}} + k_i}{2m} \right)^2 \right] - \left[\frac{\Sigma_{\text{in}}}{2m} - \left(\frac{\Sigma_{\text{tot}}}{2m} \right)^2 - \left(\frac{k_i}{2m} \right)^2 \right]$$

- Σ_{in} : somme des poids (ou nombre) des arêtes *internes* de la communauté C .
- Σ_{tot} : somme des degrés des nœuds de la communauté C .
- k_i : degré total du nœud i (poids total de ses arêtes).
- $k_{i,\text{in}}$: somme des poids des arêtes reliant le nœud i à des nœuds de la communauté C .
- m : nombre total d'arêtes (ou somme totale des poids) dans le graphe.

Algorithme de Louvain : Étapes

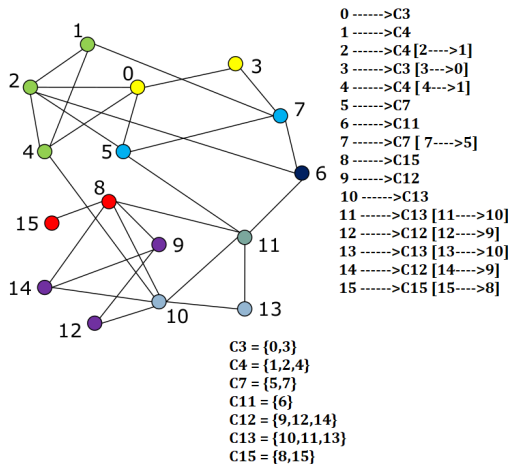
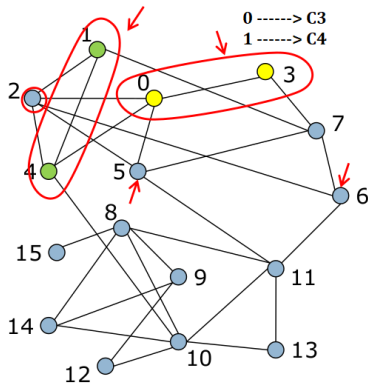
✎ Soient les communautés suivantes :

$$C_0 = \{0\}, C_1 = \{1\}, C_2 = \{2\}, C_3 = \{3\}, C_4 = \{4\}, C_5 = \{5\}, C_6 = \{6\}, C_7 = \{7\}, C_8 = \{8\}$$
$$C_9 = \{9\}, C_{10} = \{10\}, C_{11} = \{11\}, C_{12} = \{12\}, C_{13} = \{13\}, C_{14} = \{14\}, C_{15} = \{15\},$$



✎ Attention : la sélection du nœud de départ est aléatoire ; dans cet exemple, on a choisi de commencer par le nœud 0.

Algorithme de Louvain : Étapes



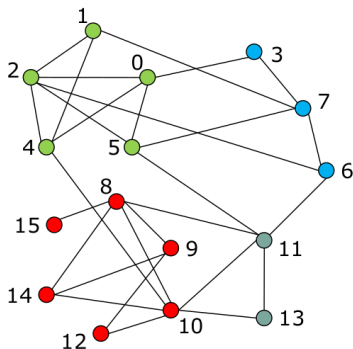
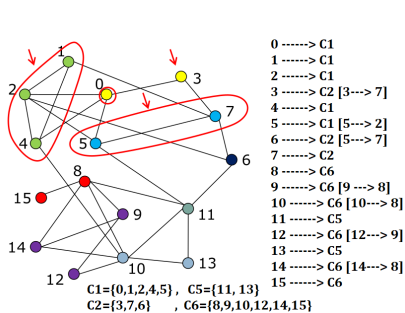
🔁 Répéter ce processus jusqu'à ce qu'aucun déplacement n'améliore plus la modularité (stabilisation $\Delta Q = 0$).

Algorithme de Louvain : Étapes

Les communautés trouvées dans le premier parcours du graphe sont :

$$C_0 = \{0, 3\}, C_1 = \{1, 2, 4\}, C_2 = \{5, 7\}, C_3 = \{6\}, C_4 = \{9, 12, 14\}$$

$$C_5 = \{10, 11, 13\}, C_6 = \{8, 15\}$$



Phase ① : Optimisation locale de la modularité

① Initialiser

- Chaque nœud forme sa propre communauté

② Pour chaque nœud (dans un ordre aléatoire) :

- Calculer le gain de modularité (ΔQ) si on le déplace vers la communauté de chacun de ses voisins
- Déplacer le nœud vers la communauté qui donne le ΔQ le plus positif
- Si aucun ΔQ n'est positif : le nœud reste dans sa communauté actuelle

③ Répéter

- Recommencer le processus pour tous les nœuds
- S'arrêter quand plus aucun déplacement n'améliore la modularité

Phase ② : Agrégation du graphe

✍ La deuxième phase ne sert qu'à créer un nouveau graphe où :

- chaque communauté obtenue à la **phase 1** devient un seul super-nœud,
- les poids des arêtes entre ces super-nœuds sont recalculés,
- puis la phase 1 est appliquée de nouveau sur ce graphe réduit.

✍ Création des arêtes entre super-nœuds

Pour chaque paire de communautés C_i et C_j trouvées en phase 1 :

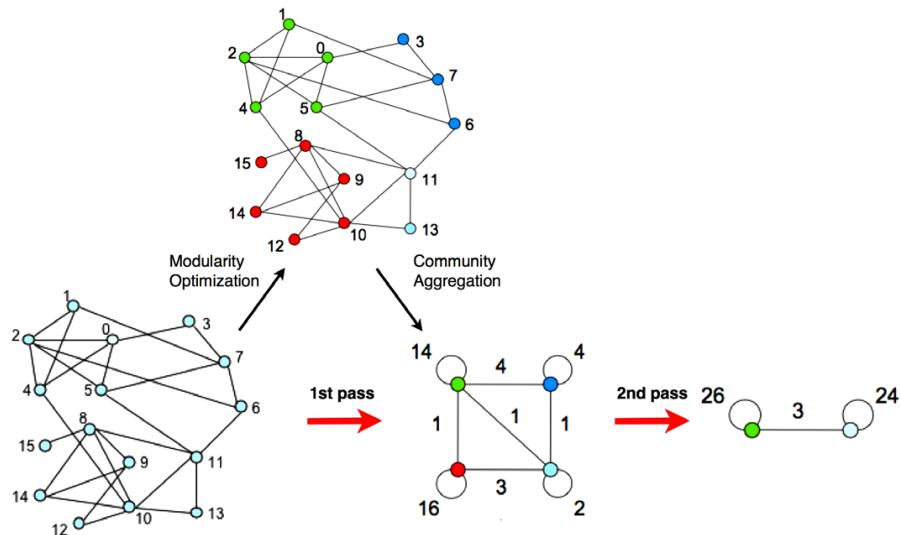
- on calcule la somme de tous les poids des arêtes entre les nœuds de C_i et ceux de C_j .
- cette somme devient le poids de l'arête entre le super-nœud C_i et le super-nœud C_j .
- Si le graphe n'est pas pondéré → on compte simplement le nombre d'arêtes.

✍ Création des boucles internes (self-loops)

Pour chaque communauté C_i trouvé en phase 1 :

- on calcule la somme des poids des arêtes à l'intérieur de la communauté (arêtes internes),
- ce total devient un self-loop (boucle) sur le super-nœud C_i .
- Ces self-loops sont importants car ils représentent la densité interne de la communauté.

Algorithme de Louvain : Étapes



Les phases 1 et 2 sont exécutées de manière itérative jusqu'à ce que la modularité (Q) ne puisse plus être améliorée.

📌 Avantages

- Très rapide : $O(m)$
- Capable de traiter des graphes très grands
- Supporte naturellement les graphes pondérés
- Méthode hiérarchique (multi-niveaux)
- Simple à implémenter
- Résultats souvent très bons en pratique
- Très utilisé et bien validé dans la littérature

📌 Inconvénients

- Limite de résolution : peut rater les petites communautés
- Non déterministe (dépend de l'ordre de parcours)
- Peut converger vers un optimum local
- Agrégation parfois trop grossière au niveau supérieur

✍ Les méthodes comme Louvain, Girvan-Newman, et d'autre cherchent à maximiser la modularité Q .

✍ Cependant, la maximisation de Q présente plusieurs problèmes :

- **Résolution limitée (resolution limit)** : en 2007, Santo Fortunato et Marc Barthélemy ont démontré que les algorithmes basés sur l'optimisation de la modularité ne peuvent pas détecter les petites communautés dans les grands graphes, en particulier celles dont le nombre d'arêtes internes m_c est inférieur à $\sqrt{2M}$, où M est le nombre total d'arêtes du graphe
- **Paysage de solutions multiples** : la fonction Q possède de nombreux maxima locaux, $\rightarrow$ résultats différents selon l'initialisation.
- **Pas de garantie d'optimalité** : la solution trouvée n'est souvent pas la vraie maximisation globale.
- **Instabilité** : petites variations dans le graphe peuvent changer fortement les communautés obtenues.
- **Modularité biaisée vers les grosses communautés** : tendance à fusionner des groupes pour augmenter artificiellement Q .

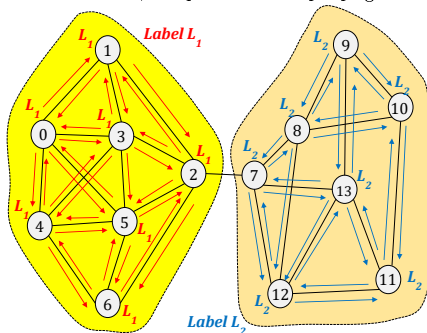
Méthode de propagation d'étiquettes : Label propagation

✎ L'algorithme de propagation d'étiquettes (Label Propagation) est une méthode de détection de communautés dans les graphes, proposée en 2007 par Usha Nandini Raghavan, Réka Albert et Soundar Kumara.

✎ Chaque nœud commence avec un label unique. À chaque itération, un nœud adopte le label le plus fréquent parmi ses voisins (vote majoritaire). Les labels se propagent et, après plusieurs itérations, des régions du graphe partagent le même label → communautés.

✎ **Principe:** Les communautés sont des groupes où :

- les nœuds ont beaucoup d'arêtes entre eux, ce qui permet au même label de circuler fortement à l'intérieur ;
- il y a peu d'arêtes vers l'extérieur, ce qui limite la propagation des autres labels.



méthode de Label propagation : étapes

✍ L'algorithme de propagation d'étiquettes repose sur une dynamique locale où les labels circulent dans le réseau jusqu'à générer des groupes cohérents.

- ➊ **Initialisation** : chaque nœud reçoit une étiquette unique.
- ➋ **Propagation** : à chaque itération, chaque nœud observe les labels de ses voisins et adopte celui qui apparaît le plus fréquemment dans son voisinage.
- ➌ **Mise à jour** : les labels se propagent progressivement dans le graphe, et des zones où un même label domine commencent à se former.
- ➍ **Convergence** : le processus s'arrête lorsque les labels ne changent plus ; les nœuds partageant le même label final constituent une communauté.

Soit un graphe non orienté : $G = (V, E)$ où V est l'ensemble des nœuds et E l'ensemble des arêtes. On note :

- $N(v)$: l'ensemble des voisins du nœud v telque $N(v) = \{u \in V \mid (u, v) \in E\}$.
- $L_v^{(t)}$: le label du nœud v à l'itération t .

Étape d'initialisation

- Chaque nœud reçoit une étiquette initiale distincte.
- Formulation mathématique :

$$L_v^{(0)} = v, \quad \forall v \in V$$

C'est-à-dire qu'à l'état initial (itération 0), le label du nœud v correspond à son identifiant.

Étape de propagation et de mise à jour des labels

✎ Dans cette étape, chaque nœud observe les étiquettes de ses voisins et met à jour son label en choisissant celui qui est le plus représenté dans son voisinage.

- Pour chaque nœud v , on collecte les labels de ses voisins :

$$S_v^{(t-1)} = \{ L_u^{(t-1)} \mid u \in N(v) \}$$

- Puis on calcule, pour chaque label ℓ , sa fréquence dans le voisinage :

$$f_v^{(t)}(\ell) = \left| \{ u \in N(v) \mid L_u^{(t-1)} = \ell \} \right|$$

- Le nœud adopte ensuite le label le plus fréquent :

$$L_v^{(t)} = \arg \max_{\ell} f_v^{(t)}(\ell)$$

Étape de propagation et de mise à jour des labels

✎ Dans le cas d'égalité (tie-breaking) où il existe plusieurs labels ℓ maximisant la fréquence:

$$\arg \max_{\ell} f_v^{(t)}(\ell) = \{\ell_1, \ell_2, \dots\}$$

On choisit :

- soit aléatoirement un label parmi les plus fréquents ;
- soit selon une règle déterministe (par exemple, choisir le plus petit label).

✎ Pour la mise à jour des labels, il existe deux méthodes principales :

• Mise à jour synchrone :

- Tous les nœuds mettent à jour leur label simultanément.
- Le calcul utilise uniquement les labels de l'itération précédente $L^{(t-1)}$.
- Peut entraîner des oscillations.

• Mise à jour asynchrone :

- Les nœuds sont mis à jour un par un, dans un ordre souvent aléatoire.
- Dès qu'un label est modifié, il peut être utilisé immédiatement par les nœuds suivants.
- Converge plus rapidement et plus stable (version la plus utilisée).

méthode de Label propagation : étapes

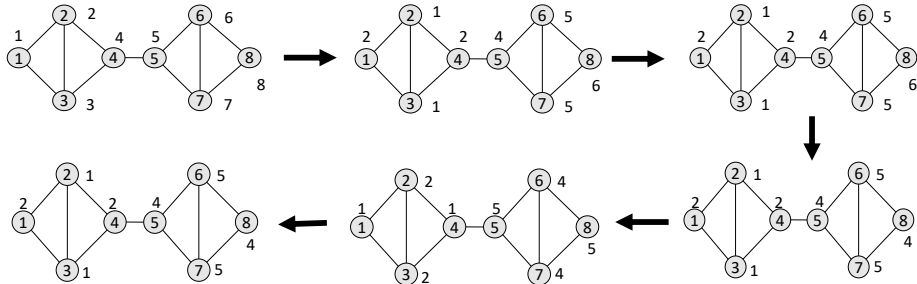
Étape de convergence

✎ La convergence de l'algorithme Label Propagation dépend du mécanisme de propagation utilisé : la version synchrone ne garantit pas la convergence et peut osciller, tandis que la version asynchrone converge toujours vers un état stable.

► Pourquoi la version synchrone peut ne pas converger ?

Lorsque les mises à jour sont simultanées :

- Les nœuds prennent des décisions sans tenir compte des changements faits par leurs voisins au même instant.
- Cela peut provoquer des modifications contradictoires.
- Si la structure du graphe est symétrique ou ambiguë, les labels peuvent alterner d'une itération à l'autre (**Le phénomène d'oscillation**).



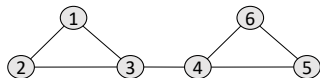
Il y a une alternance entre le label porté par {1,4} et celui porté par {2,3} (et de la même manière entre {5,8} et {6,7}). Cette alternance peut se poursuivre indéfiniment tant que la structure locale du graphe reste inchangée.

méthode de Label propagation : étapes

➡ Pourquoi la version asynchrone converge toujours ?

- En version asynchrone, le Label Propagation converge toujours vers un état stable, car chaque mise à jour est immédiatement prise en compte et empêche les oscillations.
- Si tu mets à jour un nœud et qu'il prend le label "2", puis tu mets à jour son voisin juste après, ce voisin voit déjà le label "2" et l'adopte à son tour. Ainsi, le label "2" se renforce immédiatement.

🔔 **ATTENTION** : la convergence de la version asynchrone est garantie, mais la solution obtenue n'est pas unique. Elle varie en fonction de l'ordre de mise à jour des nœuds.



Avec l'ordre : 6 → 5 → 4 → 1 → 2 → 3

→ Les deux triangles se stabilisent séparément

→ Résultat final : **2 communautés {1, 2, 3} et {4, 5, 6}**

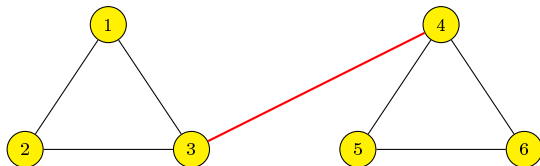
Avec l'ordre : 1 → 2 → 3 → 4 → 5 → 6

→ Le triangle gauche se stabilise d'abord et son label se propage ensuite vers l'autre .

→ Résultat final : **1 seule communauté {1, 2, 3, 4, 5, 6}**

La convergence en LPA peut être contrôlée de deux façons : soit en vérifiant que les labels ne changent plus d'une itération à l'autre, soit en fixant une limite maximale d'itérations pour arrêter l'algorithme.

méthode de Label propagation (version synchrone) : exemple



Initialisation (Itération 0) : Chaque nœud reçoit son propre label.

Labels initiaux : $\{1:'1' \ 2:'2' \ 3:'3' \ 4:'4' \ 5:'5' \ 6:'6'\}$

Itération 1 (mise à jour synchrone) : On choisit le label le plus petit en cas d'égalité.

- 1 : voisins $\{2:'2', 3:'3'\}$ → égalité → **'2'**
- 2 : voisins $\{1:'1', 3:'3'\}$ → égalité → **'1'**
- 3 : voisins $\{1:'1', 2:'2', 4:'4'\}$ → plus petit → **'1'**
- 4 : voisins $\{3:'3', 5:'5', 6:'6'\}$ → plus petit → **'3'**
- 5 : voisins $\{4:'4', 6:'6'\}$ → égalité → **'4'**
- 6 : voisins $\{4:'4', 5:'5'\}$ → égalité → **'4'**

État après It1 :

1:'2', 2:'1', 3:'1', 4:'3', 5:'4', 6:'4'

Itération 2 (lecture de It1)

- 1 : voisins $\{2:'1', 3:'1'\}$ → majorité → **'1'**
- 2 : voisins $\{1:'2', 3:'1'\}$ → plus petit → **'1'**
- 3 : voisins $\{1:'2', 2:'1', 4:'3'\}$ → plus petit → **'1'**
- 4 : voisins $\{3:'1', 5:'4', 6:'4'\}$ → majorité → **'4'**
- 5 : voisins $\{4:'3', 6:'4'\}$ → plus petit → **'3'**
- 6 : voisins $\{4:'3', 5:'4'\}$ → plus petit → **'3'**

État après It2 :

1:'1', 2:'1', 3:'1', 4:'4', 5:'3', 6:'3'

Itération 3 (lecture de It2)

- 1 : voisins $\{1,1\} \rightarrow '1'$
- 2 : voisins $\{1,1\} \rightarrow '1'$
- 3 : voisins $\{1,1,4\} \rightarrow \text{majorité} \rightarrow '1'$
- 4 : voisins $\{1:'1', 5:'3', 6:'3'\} \rightarrow \text{majorité} \rightarrow '3'$
- 5 : voisins $\{4:'4', 6:'3'\} \rightarrow \text{plus petit} \rightarrow '3'$
- 6 : voisins $\{4:'4', 5:'3'\} \rightarrow \text{plus petit} \rightarrow '3'$

État après It3 :

1:'1', 2:'1', 3:'1', 4:'3', 5:'3', 6:'3'

Itération 4 (lecture de It3)

- 1 : voisins $\{1,1\} \rightarrow '1'$
- 2 : voisins $\{1,1\} \rightarrow '1'$
- 3 : voisins $\{1,1,4\} \rightarrow \text{majorité} \rightarrow '1'$
- 4 : voisins $\{1:'1', 5:'3', 6:'3'\} \rightarrow \text{majorité} \rightarrow '3'$
- 5 : voisins $\{4:'4', 6:'3'\} \rightarrow \text{plus petit} \rightarrow '3'$
- 6 : voisins $\{4:'4', 5:'3'\} \rightarrow \text{plus petit} \rightarrow '3'$

État après It4 :

1:'1', 2:'1', 3:'1', 4:'3', 5:'3', 6:'3'

➡ **Convergence**

➡ $C_1 = \{1, 2, 3\}$

➡ $C_2 = \{4, 5, 6\}$

méthode de Label propagation (version asynchrone) : exemple

Initialisation (It0)

Chaque nœud reçoit son propre label : 1:'1' 2:'2' 3:'3' 4:'4' 5:'5' 6:'6'

Ordre d'update choisi : $6 \rightarrow 5 \rightarrow 4 \rightarrow 1 \rightarrow 2 \rightarrow 3$.

Mise à jour de 6

Voisins = {4:'4', 5:'5'} $\rightarrow$ égalité $\rightarrow$ plus petit = '4'.

État :

1:'1' 2:'2' 3:'3' 4:'4' 5:'5' 6:'4'

Mise à jour de 5

Voisins = {4:'4', 6:'4'} $\rightarrow$ majorité = '4'.

État :

1:'1' 2:'2' 3:'3' 4:'4' 5:'4' 6:'4'

Mise à jour de 4

Voisins = {3:'3', 5:'4', 6:'4'} $\rightarrow$ majorité = '4'.

État :

1:'1' 2:'2' 3:'3' 4:'4' 5:'4' 6:'4'

méthode de Label propagation (version asynchrone) : exemple

Mise à jour de 1

Voisins = $\{2: '2', 3: '3'\} \rightarrow \text{égalité} \rightarrow \text{plus petit} = '2'$.

État :

1: '**2**' 2: '2' 3: '3' 4: '4' 5: '4' 6: '4'

Mise à jour de 2

Voisins = $\{1: '2', 3: '3'\} \rightarrow \text{égalité} \rightarrow '2'$.

État :

1: '2' 2: '**2**' 3: '3' 4: '4' 5: '4' 6: '4'

Mise à jour de 3

Voisins = $\{1: '2', 2: '2', 4: '4'\} \rightarrow \text{majorité} = '2'$.

État final : 1: '2' 2: '2' 3: '**2**' 4: '4' 5: '4' 6: '4'

Convergence (état stable)

Communauté 1 : $\{1,2,3\} \rightarrow \text{label } '2'$

Communauté 2 : $\{4,5,6\} \rightarrow \text{label } '4'$

Remarque :

L'ordre choisi ($6 \rightarrow 5 \rightarrow 4 \rightarrow 1 \rightarrow 2 \rightarrow 3$) stabilise d'abord le triangle droit, ce qui empêche toute fusion entre les deux communautés.

Avantages et inconvénients du Label Propagation (LPA)

Avantages :

- Très rapide : complexité quasi linéaire $O(k.m)$.
- Aucun paramètre à définir, algorithme non supervisé.
- Simple à implémenter et basé sur une règle locale intuitive.
- Efficace pour les graphes très grands.
- Peut détecter des communautés naturelles lorsque les frontières sont fortes.

Inconvénients :

- Résultat non unique : dépend du tie-break et de l'ordre de mise à jour.
- Pas de garantie de convergence en mode synchrone (oscillations possibles).
- Sensible à l'ordre en mode asynchrone : résultats variables.
- Peut fusionner plusieurs communautés en une seule si les frontières sont faibles.
- Pas de fonction objective claire à optimiser.
- Instabilité : petites modifications du graphe influencent fortement le résultat.

Méthodes les plus connues pour la détection de communautés

📌 De nombreuses méthodes ont été proposées pour la détection de communautés ; le tableau suivant présente les approches les plus connues et les plus utilisées dans la littérature.

Méthodes	Famille / Catégorie
Louvain	Optimisation de la modularité
Leiden	Optimisation de la modularité (amélioration de Louvain)
FastGreedy	Optimisation de la modularité (greedy hiérarchique)
Infomap	Méthodes basées sur les flots / random walks
Walktrap	Random walks (distances de marche aléatoire)
Label Propagation (LPA / FLPA)	Propagation d'étiquettes locales
Spinglass	Modèles physiques (Potts / Ising)
Stochastic Block Model (SBM)	Modèles probabilistes / génératifs
Girvan–Newman	Méthodes divisives (suppression d'arêtes à forte betweenness)
Spectral Clustering	Méthodes spectrales (Laplacien du graphe)

✎ Lorsque l'on détecte des communautés dans un graphe (réseaux sociaux, interaction entre utilisateurs, réseaux biologiques...), on doit évaluer si la partition trouvée est cohérente, qualitative et significative.

✎ Il existe deux grandes familles d'évaluations :

① Les mesures internes (aucune vérité terrain $\rightarrow$ on juge la qualité structurelle du partitionnement)

- Modularity (Q)
- Conductance
- Density / Cohesion
- Couverture / Coverage
- Expansion / Cut Ratio
-

② Les mesures externes (on connaît la vraie communauté $\rightarrow$ on compare)

- Normalized Mutual Information (NMI)
- Adjusted Rand Index (ARI)
- Purity / Accuracy
- F1-score
-

- **Modularity** $Q(P)$

$$Q(P) = \frac{1}{2m} \sum_{C \in P} \sum_{i,j \in C} \left(A_{ij} - \frac{k_i k_j}{2m} \right)$$

Bonne valeur : 0.3 à 0.7 (bonne partition)

- **Conductance globale** $\phi(P)$

$$\phi(C) = \frac{\text{cut}(C, \bar{C})}{\min(\text{vol}(C), \text{vol}(\bar{C}))}$$

$$\phi(P) = \frac{1}{|P|} \sum_{C \in P} \phi(C)$$

Bonne valeur : $\phi(P) \approx 0$

- **Coverage (couverture)**

$$\text{Coverage}(P) = \frac{\sum_{C \in P} |E_C|}{|E|}$$

Bonne valeur : proche de 1

-
- E_C : ensemble des arêtes internes à la communauté C
 - $\text{cut}(C, \bar{C})$: nombre d'arêtes reliant C à son complément

- C : une communauté dans la partition P
- $\bar{C}$: le complément de C dans le graphe
- $m = |E|$: nombre total d'arêtes du graphe
- $\text{vol}(C) = \sum_{u \in C} k_u$: volume de C (somme des degrés)

- **Performance de la partition**

$$\text{Perf}(P) = \frac{|\{(i, j) \in E : c_i = c_j\}| + |\{(i, j) \notin E : c_i \neq c_j\}|}{n(n-1)/2}$$

Bonne valeur : proche de 1

- **Partition Density $D(P)$**

$$D(P) = \frac{2}{|E|} \sum_{C \in P} \frac{|E_C| - (|C| - 1)}{(|C| - 2)(|C| - 1)}$$

Bonne valeur : valeurs élevées

- **Cut Ratio global**

$$CR(P) = \frac{1}{|P|} \sum_{C \in P} \frac{\text{cut}(C, \bar{C})}{|C| \cdot |\bar{C}|}$$

Bonne valeur : proche de 0

✎ Les métriques de comparaison (évaluation externe) consistent à mesurer la similarité entre deux partitions d'un même graphe :

- la **partition X** générée par l'algorithme ;
- la **partition Y** considérée comme la référence (ground truth).

✎ **Normalized Mutual Information (NMI)**

$$NMI(X, Y) = \frac{2I(X, Y)}{H(X) + H(Y)}$$

Interprétation : La NMI mesure la similarité informationnelle entre deux partitions ; elle varie de 0 (aucune correspondance) à 1 (correspondance parfaite)

✎ **Adjusted Rand Index (ARI)**

$$ARI(X, Y) = \frac{2(a_{11}a_{00} - a_{10}a_{01})}{(a_{11} + a_{01})(a_{00} + a_{01}) + (a_{11} + a_{10})(a_{00} + a_{10})}$$

- a_{11} : couples de nœuds dans la même communauté dans X et dans Y .
- a_{00} : couples de nœuds dans des communautés différentes dans X et dans Y .
- a_{10} : couples de nœuds dans la même communauté dans X mais dans des communautés différentes dans Y .
- a_{01} : couples de nœuds dans des communautés différentes dans X mais dans la même communauté dans Y .

Interprétation : L'ARI mesure la similarité entre deux partitions en corrigeant l'effet du hasard ; il varie de 0 (niveau aléatoire) à 1 (accord parfait).

📖 Purity

$$Purity(X, Y) = \frac{1}{n} \sum_{C_i \in X} \max_{Y_j \in Y} |C_i \cap Y_j|$$

Interprétation : La pureté mesure à quel point chaque communauté de X correspond à une communauté réelle de Y ; elle varie entre 0 et 1, avec des valeurs proches de 1 indiquant une bonne correspondance.

F1-score

$$Precision(C_i, Y_j) = \frac{|C_i \cap Y_j|}{|C_i|}$$

$$Recall(C_i, Y_j) = \frac{|C_i \cap Y_j|}{|Y_j|}$$

$$F_1(C_i, Y_j) = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

Interprétation : Le F1-score mesure la similarité entre une communauté C_i de X et une communauté Y_j de Y ; il varie entre 0 (mauvaise correspondance) et 1 (correspondance parfaite).

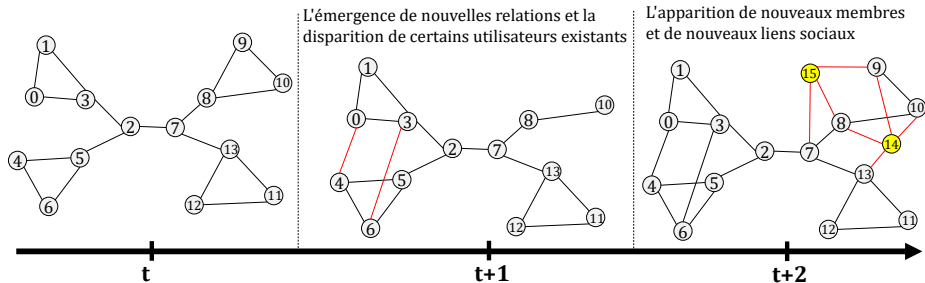
- ARI → le plus utilisé, corrige le hasard, bonnes valeurs proches de 1
- NMI → basé information, invariant au nombre de clusters
- Purity / F1 → utiles mais biaisés si beaucoup de communautés

Détection de communautés dans les réseaux sociaux dynamiques

✎ Dans de nombreux réseaux réels, les interactions ne sont pas stable et fixe : elles évoluent au fil du temps. Les groupes d'individus se forment, disparaissent, se fusionnent ou se divisent.

✎ Ainsi, la structure des communautés change automatiquement dès qu'il y a de nouvelles interactions ou modifications dans les réseaux sociaux.

✎ La détection de communautés dynamique permet justement d'analyser cette évolution en identifiant non seulement les communautés à chaque instant, mais aussi la manière dont elles changent.



➡ Comment développer des méthodes qui considèrent les changements qui se produisent dans les réseaux sociaux ?

un réseau social dynamique peut être modélisé comme une suite de graphes :

$$\mathcal{G} = \{G_1, G_2, \dots, G_T\},$$

où chaque instantané est défini par :

$$G_t = (V_t, E_t), \quad t = 1, \dots, T,$$

avec V_t l'ensemble des nœuds et E_t l'ensemble des liens au temps t . L'évolution du réseau est alors décrite par les changements entre ces graphes au fil du temps.

✎ En plus de la représentation classique par une suite de graphes, il existe plusieurs autres façons de modéliser un réseau dynamique, chacune mettant l'accent sur un aspect différent de l'évolution temporelle.

- Graphes temporels : les arêtes sont horodatées.
- Modèles à intervalles : chaque lien existe sur une période $[t_{start}, t_{end}]$
- Réseaux multi-couches : une couche par période ou type d'interaction.
- Modèles continus : les liens varient comme une fonction du temps.
- Modèles probabilistes : l'évolution du réseau est décrite par des lois de probabilité (ex. DSBM).

✎ Il existe plusieurs méthodes pour la détection des communautés dans les réseaux sociaux dynamiques. En général, ces méthodes se regroupent autour de quatre grandes approches :

- **Approche par instantanés uniformes successifs** : le réseau est découpé en instantanés de durée fixe, et la détection des communautés est effectuée séparément sur chaque snapshot.
- **Approche par instantanés consécutifs, deux à deux interdépendants** : la détection à l'instant t tient compte à la fois du snapshot actuel et des communautés trouvées à l'instant $t - 1$ afin d'assurer une continuité temporelle.
- **Approche par instantanés multi-pas** : chaque partition dépend non seulement de l'instant précédent, mais également de plusieurs instantanés antérieurs, ce qui permet une prise en compte plus profonde de l'historique.
- **Approche incrémentale** : la structure des communautés est mise à jour à chaque modification du réseau (ajout ou suppression d'arêtes ou de nœuds) sans recalcul complet de la partition.

Approche ❶ : Détection indépendante et appariement (Independent Community Detection and Matching)

✍ La première solution pour analyser un réseau dynamique consiste à exploiter directement les méthodes de détection de communautés **statiques**, en les appliquant sur chaque état du réseau au fil du temps.

✍ Principe :

- Le réseau dynamique est découpé en une suite de graphes statiques :

$$\{G_1, G_2, \dots, G_T\}$$

- À chaque instantané G_t , on applique un algorithme classique (Louvain, Infomap, LPA, Spinglass, etc.).
- On obtient ainsi des communautés $C_1, C_2, \dots, C_T$ qui seront ensuite comparées pour suivre leur évolution.

✍ Chaque instantané est traité indépendamment des autres.

Snapshot 1 $\rightarrow$ communautés $C_1 = \{C_1^1, C_1^2, \dots, C_1^{k_1}\}$

Snapshot 2 $\rightarrow$ communautés $C_2 = \{C_2^1, C_2^2, \dots, C_2^{k_2}\}$

Snapshot 3 $\rightarrow$ communautés $C_3 = \{C_3^1, C_3^2, \dots, C_3^{k_3}\}$

...

Snapshot n $\rightarrow$ communautés $C_n = \{C_n^1, C_n^2, \dots, C_n^{k_n}\}$

Approche ❶ : Détection indépendante et appariement (Independent Community Detection and Matching)

➤ Après avoir détecté les communautés dans chaque snapshot, on cherche à faire correspondre les communautés entre deux instants successifs. Par exemple : La communauté A à l'instant t correspond-elle à la communauté B à l'instant $t+1$?

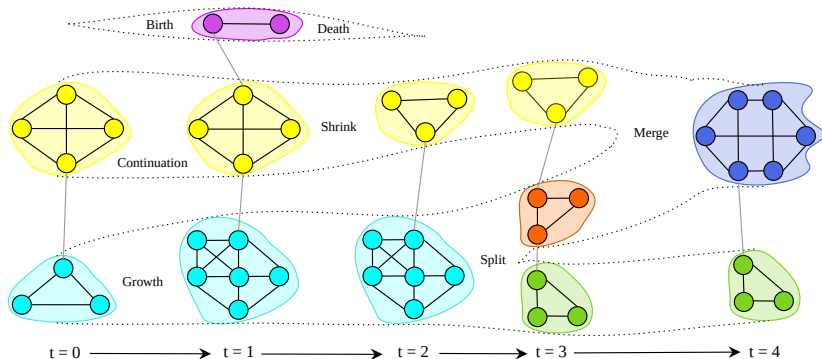
➤ Pour établir cette correspondance, on utilise souvent des mesures de similarité comme :

Mesure	Formule	Description
Jaccard	$S(A, B) = \frac{ A \cap B }{ A \cup B }$	Mesure la proportion d'éléments communs par rapport à l'ensemble des éléments.
Coefficient de recouvrement	$S(A, B) = \frac{ A \cap B }{\min(A , B)}$	Indique dans quelle mesure la plus petite communauté est incluse dans la plus grande.
Intersection simple	$S(A, B) = A \cap B $	Compte uniquement le nombre de nœuds en commun, utilisé pour le matching biparti.
Sorensen-Dice	$S(A, B) = \frac{2 A \cap B }{ A + B }$	Similarité équilibrée, moins sensible aux différences de taille que Jaccard.
Cosinus	$S(A, B) = \frac{ A \cap B }{\sqrt{ A B }}$	Basé sur les vecteurs indicateurs, adapté lorsque les communautés ont des tailles variées.

Approche ① : Détection indépendante et appariement (Independent Community Detection and Matching)

✎ Une communauté peut subir plusieurs types de changements :

- Apparition (birth) : une nouvelle communauté se forme.
- Disparition (death) : la communauté cesse d'exister.
- Croissance (growth) : la communauté gagne de nouveaux nœuds.
- Réduction (shrink) : la communauté perd certains nœuds.
- Fusion (merge) : deux ou plusieurs communautés se regroupent en une seule.
- Scission (split) : une communauté se divise en plusieurs communautés.
- Continuation (survival) : la communauté reste globalement la même au fil du temps.



Approche ❶ : Détection indépendante et appariement (Independent Community Detection and Matching)

- **Continuation** : $\exists C_{t+1}^j$ tel que $J(C_t^i, C_{t+1}^j) \geq \theta$.
- **Fusion** : plusieurs communautés de t mènent à une même communauté de $t + 1$:

$$J(C_t^{i1}, C_{t+1}^j) \geq \theta, J(C_t^{i2}, C_{t+1}^j) \geq \theta, \dots$$

- **Scission (split)** : une communauté de t se divise en plusieurs à $t + 1$:

$$J(C_t^i, C_{t+1}^{j1}) \geq \theta, J(C_t^i, C_{t+1}^{j2}) \geq \theta, \dots$$

- **Apparition (birth)** : une communauté C_{t+1}^j est nouvelle si $\forall i, J(C_t^i, C_{t+1}^j) < \theta$.

- **Disparition (death)** : une communauté C_t^i disparaît si $\forall j, J(C_t^i, C_{t+1}^j) < \theta$.

- **Croissance (growth)** : continuation plus augmentation de taille :

$$J(C_t^i, C_{t+1}^j) \geq \theta \quad \text{et} \quad |C_{t+1}^j| > |C_t^i|.$$

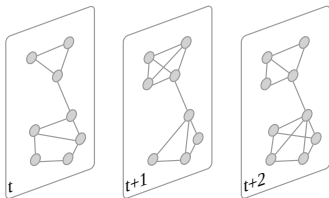
- **Réduction (shrink)** : continuation plus diminution de taille :

$$J(C_t^i, C_{t+1}^j) \geq \theta \quad \text{et} \quad |C_{t+1}^j| < |C_t^i|.$$

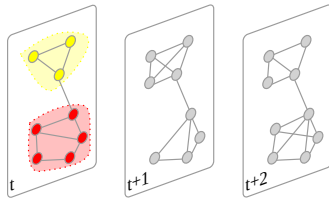
Détection de communautés dans les réseaux sociaux dynamiques

Approche ② : pproche par instantanés consécutifs, deux à deux interdépendants

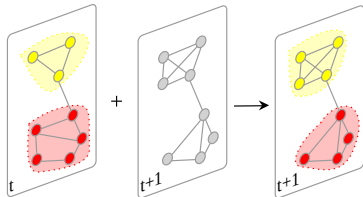
🔗 L'approche **Dependent Community Detection** consiste à détecter les communautés dans un réseau dynamique en tenant compte explicitement des résultats de l'instant précédent. Contrairement à l'approche indépendante, où chaque snapshot est traité séparément, ici la détection à l'instant t dépend directement de ce qui a été trouvé à l'instant $t-1$.



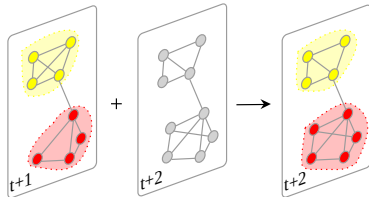
(a) Dynamic network consisting of three snapshots.



(b) Community structure in the first snapshot.



(c) Community detection at $t+1$ using graph of $t+1$ and community structure of t .



(d) Community detection at $t+2$ using graph of $t+2$ and community structure of $t+1$.

Approche ② : pproche par instantanés consécutifs, deux à deux interdépendants

📖 Comment cela fonctionne ?

① Au temps $t=1$

On applique un algorithme classique (Louvain, Infomap, Lieden etc.).

② Au temps $t \geq 2$

La partition au temps t dépend de deux informations:

- la structure du graphe G_t
- la partition précédente C_{t-1}

③ Optimisation double : qualité + stabilité

La plupart des méthodes dépendantes cherchent à maximiser une fonction du type :

$$\text{Score}(t+1) = \alpha Q(G_{t+1}, C_{t+1}) + (1 - \alpha) S(C_{t+1}, C_t), \quad \alpha \in [0, 1]$$

- $Q(G_{t+1}, C_{t+1})$: qualité de la partition C_{t+1} sur le graphe $G(t+1)$ (ex. modularité)
- $S(C_{t+1}, C_t)$: similarité entre la nouvelle partition et la partition précédente (ex. somme des Jaccard)
- α : paramètre d'équilibre entre qualité instantanée et stabilité temporelle

④ La partition maximisant la fonction $\text{Score}(t+1)$ à l'instant $t+1$ est retenue comme la meilleure partition.

Approche ② : pproche par instantanés consécutifs, deux à deux interdépendants

Algorithm 1 Détection dépendante des communautés (instantanés consécutifs)

```
1: Entrée : séquence de graphes dynamiques  $\{G_1, G_2, \dots, G_T\}$ , paramètre  $\alpha \in [0, 1]$ 
2: Sortie : partitions  $\{C_1, C_2, \dots, C_T\}$ 

3: Étape 1 : Détection initiale
4:  $C_1 \leftarrow \text{DetectCommunity}(G_1)$ 

5: for  $t = 1$  to  $T - 1$  do
6:   Étape 2 : Génération des partitions candidates
7:    $\mathcal{P} \leftarrow \text{GenerateCandidates}(G_{t+1})$ 

8:   Étape 3 : Calcul du score pour chaque partition candidate
9:   for chaque  $C$  dans  $\mathcal{P}$  do
10:     $\text{score}(C) \leftarrow \alpha \cdot Q(G_{t+1}, C) + (1 - \alpha) \cdot S(C, C_t)$ 
11:   end for

12:   Étape 4 : Sélection de la meilleure partition
13:    $C_{t+1} \leftarrow \arg \max_{C \in \mathcal{P}} \text{score}(C)$ 
14: end for

15: return  $\{C_1, C_2, \dots, C_T\}$ 
```

Approche ③ : Approche incrémentale (Incremental Community Detection)

✍ L'approche incrémentale est une méthode de détection des communautés dans les réseaux dynamiques où la partition des communautés est mise à jour progressivement, à chaque petit changement du réseau.

✍ Ce n'est donc pas une détection complète à chaque snapshot, mais une mise à jour locale et rapide.

✍ Idée principale

À chaque modification du réseau (ajout ou suppression d'un nœud ou d'un lien), on ne relance pas une détection complète des communautés : on met à jour uniquement les parties de la partition réellement affectées.

➡ **C'est comme corriger un travail déjà fait, au lieu de le refaire entièrement.**

✍ Les modifications du réseau dynamique sont représentées par quatre types d'événements :

- l'ajout d'un nœud
- la suppression d'un nœud
- l'ajout d'une arête
- la suppression d'une arête

Approche ③ : Approche incrémentale (Incremental Community Detection)

🔧 Comment ça fonctionne ?

① On commence avec une partition initiale

À l'instant $t = 1$, on applique un algorithme statique (Louvain, LPA, Infomap...) pour construire la structure communautaire initiale $C_1 = \{C_1^1, C_1^2, \dots, C_1^{k_1}\}$.

② Le réseau évolue

À l'instant $t + 1$, détecter les changements du réseau :

- ajout d'une arête
- suppression d'une arête
- arrivée d'un nouveau nœud
- disparition d'un nœud

③ Mise à jour locale

Au lieu de recalculer toutes les communautés, on analyse :

- quels nœuds sont affectés ?
- quelles communautés sont modifiées ?

Puis on ne met à jour que ces parties-là.

④ Ajustement minimal

On ajuste :

- les frontières entre communautés
- la position des nœuds concernés
- parfois la fusion ou la division locale d'une communauté

Le reste de la partition ne change pas.

Approche ③ : Approche incrémentale (Incremental Community Detection)

Algorithm 2 Détection incrémentale des communautés

```
1: Entrée : partition initiale  $C_1$ , flux de changements du graphe
2: Sortie : partition mise à jour  $C_t$ 
3: for chaque changement  $\Delta$  dans le graphe do
4:   identifier les nœuds affectés par  $\Delta$ 
5:   analyser leur communauté actuelle
6:   mettre à jour localement : déplacer, fusionner ou ajuster
7:   propager les ajustements si nécessaire
8: end for
9: return partition finale  $C_t$ 
```
